University of Kansas

Arithmetic Expression Evaluator C++

User's Manual

Version 1.1

Revision History

Date	Version	Description	Author
05/May/26	1.0	Added introduction, installation/setup, getting started, and troubleshooting	Alexander W.
07/May/26	1.1	Finished all remaining sections and updated formatting	John P.

Table of Contents

User Manual

1. Purpose

The user manual explains how to use the Arithmetic Expression Evaluator software to solve expressions. It is designed to be user-friendly and uses simple step-by-step instructions to allow the user to enter expressions, understand the results, and troubleshoot common issues.

2. Introduction

The Arithmetic Expression Evaluator is a command line application developed in C++ used for evaluating simple arithmetic expressions. Our evaluator handles both positive and negative values, addition, subtraction, multiplication, division, modulo, exponentiation, and parentheses for grouping, as well as adhering to PEMDAS.

To get started using the evaluator:

1. Clone the repo from the GitHub using 'git clone https://github.com/Austin-Haight/EECS348-project.git'
2. Go inside the new folder "EECS348-project" using 'cd' command
3. Go inside the "build" folder using 'cd' command
4. Enter 'make' in the terminal
5. Enter 'make run' in the terminal

Done!

3. Getting started

Now that you have the program compiled and running you can interact with it via command-line. The following steps can help in using the software to evaluate arithmetic expressions.

3.1 Entering expressions

To get started type an arithmetic expression and press Enter.
Examples of how to use operators with screenshots:

- Addition: 3 + 4 → 7
- Subtraction: 5 – 4 → 1
- Grouping with parentheses: 2 * (5 + 6) → 22
- Unary minus: -5 + 10 → 5
- Unary plus: -(+1) + (+2) → 1
- Division: 10 / 2 → 5
- Exponentiation: 2 ** 3 → 8
- Modulo: 8 % 3 → 2

```
Expression: 3 + 4
Input: 3 + 4
Parser output: + 3 4

Final output: 7.000000
```

```
Expression: 2 * (5+6)
Input: 2 * (5+6)
Parser output: * 2 + 5 6

Final output: 22.000000
```

Operator	Description
()	Parentheses
**	Exponentiation
*, /, %	Multiplication, Division, Modulo
+, -	Addition or Unary Plus, Subtraction or Unary Minus

3.2 Reading the results

After pressing Enter:

- If you have entered a valid arithmetic expression, the evaluator will return a **numeric result** of your expression. Also, you can see the input and parser output in the terminal.

- If you have entered an invalid arithmetic expression, the evaluator will return an **error message**.

3.3 Leaving the program

To quit out of the program at any time you can send "quit" or "exit" to the command line and your result history can be viewed as well with "history". After entering "exit", you also may enter 'make clean' to delete the build files created.

4. Advanced features

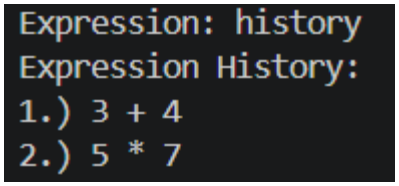
This section includes the advanced features for the Arithmetic Expression Evaluator.

4.1 History

The evaluator can keep track of past expressions entered during the current session. Users can view this any time by typing "history" into the command line. When entered, the evaluator will display a numbered list of all expressions the user has previously entered in the correct order.

Example:

If the user enters "3 * 4" and "5 * 7", and enters "history", the following will be displayed in the terminal.



5. Troubleshooting

Here are some common mistakes that often result in errors or expressions being unable to be calculated.

5.1 Unmatched parentheses

Ensure that when using parentheses they are in the form of (expression). Opening but not closing a set of parentheses as seen below will result in an invalid expression.

Examples:

- 2 * (4* 3 + 1
- 4 + 5) * 3
- ((6*3) + 1
- (4 + 3 (* 2)

5.2 Divide by zero

Division by zero is not possible so attempting to do so will result in an error from the evaluator.

Examples:

- 2/0
- (4**6)/0
- 4 + 5/0

5.3 Invalid characters

The evaluator only supports the numeric constants and the operators listed above in the "Getting Started" section of this guide. Other inputs will be treated as invalid and return an error

Examples:

- 7 + abc
- 4 ^ 2 (use ** instead)
- 1 && 0
- 5 + 2 =

5.4 Missing operands

Every operator needs two operands to be able to return a value, using less than two will result in an error as the evaluator will have too little to process your expression.

Examples:

- *5+2
- 3 +
- %4

5.5 Too many operators

Ensure that two operands are always paired with one operator.

Examples:

- 2 ++ 2 (try 2 + 2)
- %4%5
- 7***7

6. Examples

This section provides sample expressions that help you understand how to use the Arithmetic Expression Evaluator.

6.1 Basic arithmetic

Expression	Explanation	Result
3 + 4	Simple addition	7
10 – 6	Simple subtraction	4
5 * 3	Multiplication	15
20 / 4	Division	5
5 ** 2	Exponentiation	25
15 % 6	Modulo	3

6.2 Using parenthesis

Expression	Explanation	Result
2 * (5 + 6)	Parentheses first	22
(8 - 3) (2 + 1)	Multiple groups	15
(5+5) * 2	Parenthesis first	20

6.3 Unary operators

Expression	Explanation	Result
-(3 + 2)	Negates the group	-5
-(+1) + (+2)	Unary plus and minus	1

6.4 Combined expressions

Expression	Explanation	Result
3 + 4 * 2	Multiplication has higher precedence	11
2 ** 3 * 4	Exponentiation before multiplication	32

7. Glossary of terms

The following are terms used throughout the User Manual and their corresponding definitions:

Operator: A symbol that completes a mathematical action

Operand: A number used by an operator

Unary Operator: An operator that applies to a single operand

Command Line: Interface where the user can interact with the system by typing expressions to be evaluated

Compile: Converting a C++ file into an executable program

Executable: Final program file that is created after compiling the program

8. FAQ

Q1: What kinds of expressions can I enter?

You may enter any valid expression using numbers, parentheses, and the supported operators listed in the "Getting Started" section.

Q2: What happens if I type something invalid?

You will receive an error message that allows you to understand what went wrong in your input.

Q3: Does the evaluator support negative numbers?

Yes. You may enter expressions with negative numbers, such as -5 + 10.

Q4: Is PEMDAS followed in the evaluation?

Yes. Multiplication, division, and exponentiation happen before addition and subtraction unless parentheses change the order.

Q5: Can I see my history of past expressions?

Yes. Typing "history" in the command line will allow you to see past expressions.

Q6: How do I exit the evaluator when I am done?

Typing "exit" or "quit" in the command line will exit the evaluator.

For example: